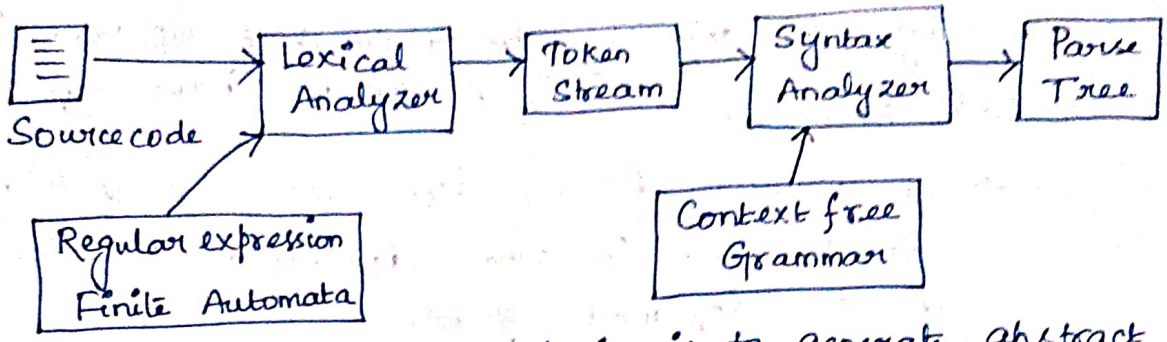
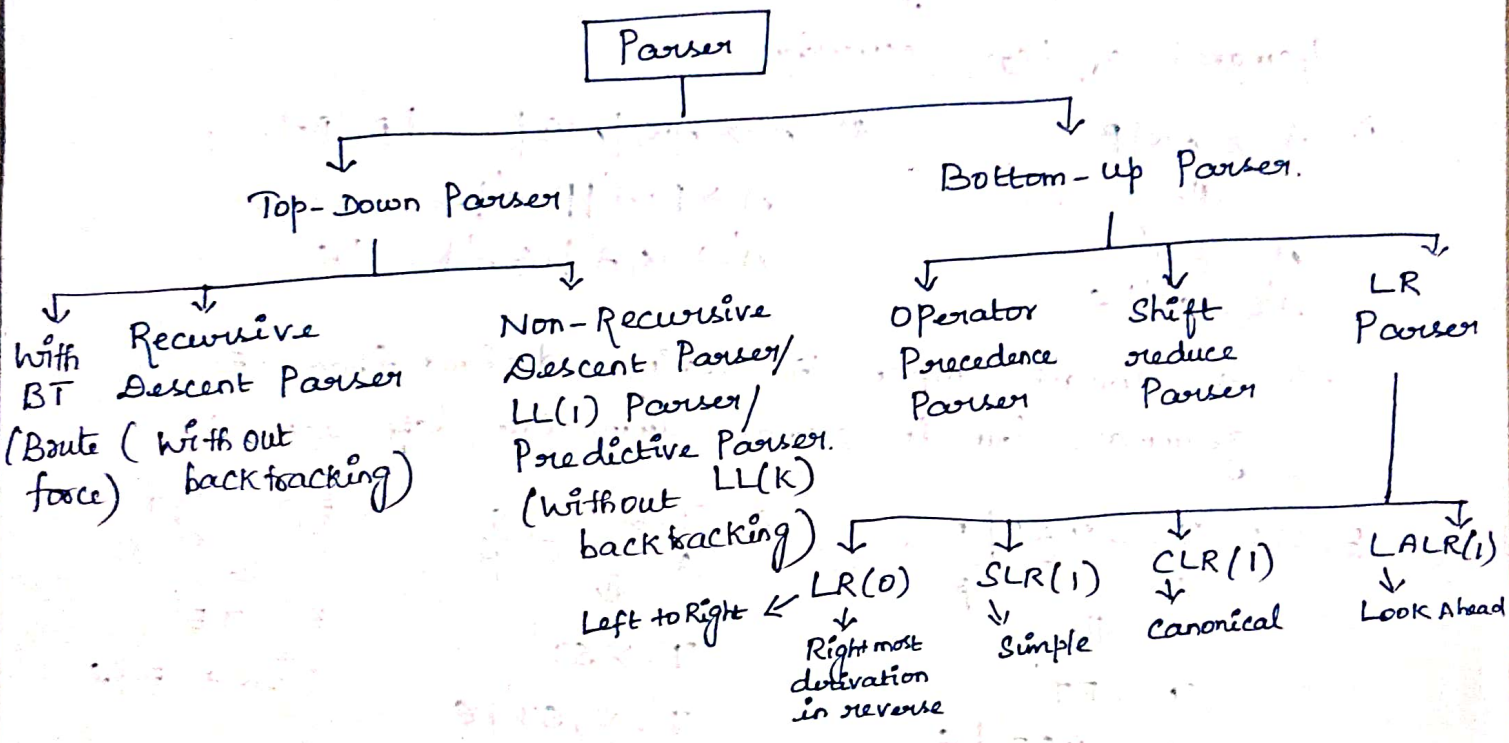
# Syntax Analysis (Parser)

→ Role of a Parser is to construct a Parse tree.

Token streams ⇒ Parse tree.




→ The main goal of syntax analysis is to generate abstract syntax tree of the source code, that reflects the grammatical structure of the program.




→ TOP-Down → Parsing starts from the highest level of the syntax tree and works its way down.

→ It uses left most derivation.

Bottom-up → Parsing starts from the lowest level and works it way up.

→ Uses right most derivation.

Top down Parser does not support left recursion & left factoring. ~~If any~~ In order to construct Top down Parser the CFG should not have left recursion

(1) Left Recursion:-

→ A grammar becomes left-recursive if it has any non-terminal 'A' whose derivation contains 'A' itself as the left most symbol.

Ex: 1) $A \to A\alpha \mid \beta$ → immediate left recursion.

2) $\left.\begin{array}{l} S \to A\alpha \mid \beta \\ A \to Sd \end{array}\right\}$ indirect left recursion.

Removal of Left recursion

1) $A \to A\alpha \mid \beta$

$\Downarrow$

$A \to \beta A'$

$A' \to \alpha A' \mid \epsilon$

2) $A \to A\alpha_1 \mid A\alpha_2 \mid \dots \mid \beta_1 \mid \beta_2 \mid \dots$

$A \to \beta_1 A' \mid \beta_2 A' \mid \dots$

$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \epsilon$

→ It doesn't impact the strings derived from the grammar, but it removes immediate left recursion.

Ex:- 1. $\underset{A}{T} \to \underset{\alpha}{T*F} \mid \underset{\beta}{F}$

$\Downarrow$

$T \to FT'$

$T' \to *FT' \mid \epsilon$

2. $\underset{A}{T} \to \underset{\alpha}{T,S} \mid \underset{\beta}{S}$

$\Downarrow$

$T \to ST'$

$T' \to ,ST' \mid \epsilon$

3. $\underset{A}{E} \to \underset{\alpha}{E+T} \mid \underset{\beta}{T}$

$\Downarrow$

$E \to TE'$

$E' \to +TE' \mid \epsilon$

4. $\underset{A}{E} \to \underset{\alpha}{E+T} \mid \underset{\beta}{ab \mid cd}$ ⇒ $E \to abE' \mid cdE'$

$E' \to +TE' \mid \epsilon$

5. $A \to (B) \mid b$

$\underset{A}{B} \to \underset{A}{B}\underset{\alpha}{xA} \mid \underset{\beta}{A}$

$\Downarrow$

$A \to (B) \mid b$

$B \to AB'$

$B' \to xAB' \mid \epsilon$

6. $S \to Aa \mid \beta$

$A \to Sd$

$\Downarrow$

$S \to Aa \mid \beta$

$A \to Aad \mid \beta d$

$\Downarrow$

$S \to Aa \mid \beta$

$A \to \beta d A'$

$A' \to adA' \mid \epsilon$

7. $T \to T*F \mid F$

$F \to (E) \mid id$

8. $S \to S0S1S \mid 01$

9. $S \to (L) \mid x$

$L \to L, S \mid S$

For multiple production:-

Ex:- 1) $\underset{A}{expr} \to \underset{A}{expr} + \underset{\alpha_1}{expr} \mid \underset{A}{expr} * \underset{\alpha_2}{expr} \mid \underset{\beta_1}{id}$

$expr \to id\, expr'$

$expr' \to +expr\, expr' \mid *expr\, expr' \mid \epsilon$

2) $\underset{A}{S} \to \underset{A\ \alpha_1}{Sx} \mid \underset{A\ \alpha_2}{Ssb} \mid \underset{\beta_1}{xS} \mid \underset{\beta_2}{a}$

$S \to xSS' \mid aS'$

$S' \to xS' \mid SbS' \mid \epsilon$

Left factoring:-

→ If the grammar has, $A \to \alpha\beta_1 \mid \alpha\beta_2$

→ if more than one grammar production rules has a common prefix string, then the topdown parser cannot make a choice to which of the production it should take to parse the string.

(Non-determinism)

$A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \cdots \mid \gamma_1 \mid \gamma_2$

$\Downarrow$

$A \to \alpha A' \mid \gamma_1 \mid \gamma_2$

$A' \to \beta_1 \mid \beta_2 \mid \cdots$

1. $S \to iEtS \mid iEtSeS \mid a$
$E \to b$

$\Downarrow$

$\underset{A}{S} \to \underset{\alpha}{iEtS} \mid \underset{\alpha}{iEtS}\underset{\beta_2}{eS} \mid \underset{\gamma_1}{a}$

$S \to iEtSS' \mid a$
$S' \to \epsilon \mid eS$

2. $\underset{A}{S} \to \underset{\alpha}{T}+S \mid \underset{\alpha}{T}$

$\Downarrow$

$S \to TS'$
$S' \to +S \mid \epsilon$

1. <u>Recursive Descent Parsing</u>
   - → Top-Down Parser
   - → Uses Recursive procedure
   - → No Backtracking
   - → No left recursion, No left factoring.

<u>Rules:-</u>

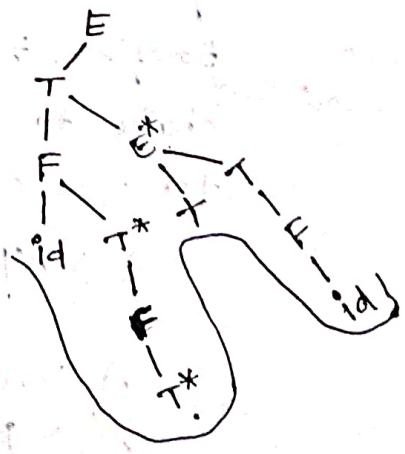
R-1 → If input is Non-Terminal, then write procedure

R-2 → If input is Terminal,
Compare Terminal with input symbol
if both are equal then increment
input pointer.

R-3 → If any Non-Terminal having multiple productions,
write all the productions in single procedure.

X R-4 → No main & No declarations.

<u>Ex:-</u> $E \to TE^*$
$E^* \to +TE^* \mid \epsilon$
$T \to FT^*$
$T^* \to *FT^* \mid \epsilon$
$F \to (E) \mid id.$



```
Procedure E()
{
    T(); →
    Estar();
}
Procedure Estar()
{
    if( input == '+')
    {   input ++;
        T();
        Estar();
    }
    else
        return;
}
```

```
Procedure T()
{
    F(); →
    Tstar();
}
Procedure Tstar()
{   if(input == '*')
    {   input ++;
        F();
        Tstar();
    }
    else
        return;
}
```

```
Procedure F() ←
{   if(input == '(')
    {   input ++;
        E();
        if(input == ')')
        {   input ++;
        }
    }
    if(input == 'id')
        input ++;
}
```

i/p String : id + id $ ⇒ id + id $ ⇒ id + id $ ⇒ id + id $

input input input input

Ex: 2 $E \rightarrow iE'$

$E' \rightarrow +iE' \mid \epsilon$

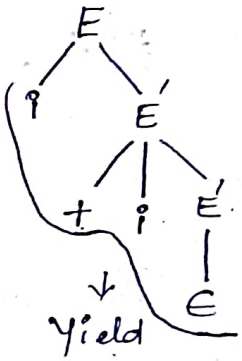
```
1. E()
2. {
3.     if ( input == 'i')
4.     {
5.         match ('i');
✓ 6.       E'();
✓ 7.   }
8. }
```

```
match(char c)
{   if( input == c)
        input = getchar();
    else
        Printf ("Error!");
}
```

i/p : i+i$

input

```
1. E'()
{   if( input == '+')
    {   match('+');
✓       match('i');
        E'();
    }
    else
        return;
}
```




```
1 main()
2 {
3     E();
4     if( input == '$')
5         Printf(" Parsing Successful");
6 }
```

# FIRST & FOLLOW Set

## To calculate FIRST Set

Consider $\alpha$ - terminal, $\beta$ - non-terminal.

If the production is, $X \to \epsilon$ then $First(X) = \{\epsilon\}$

$X \to \alpha$ then $First(X) = \{\alpha\}$

$X \to \alpha\beta_1\beta_2 \ldots$ then $First(X) = \{\alpha\}$

$X \to \beta_1\beta_2 \ldots$

$First(X) = First(\beta_1)$ if it ~~doesn't~~ doesn't have $\epsilon$

$First(X) = \{First(\beta_1)$ without $\epsilon\} \cup First(\beta_2\beta_3 \ldots)$

Ex: 1

$S \to aBDh$ $\quad First(S) = \{a\}$

$B \to cC$ $\quad First(B) = \{c\}$

$C \to bC / \epsilon$ $\quad First(C) = \{b, \epsilon\}$

$D \to EF$ $\quad First(D) = \{g, f, \epsilon\} \Rightarrow First\{E \text{ w/o } \epsilon\} \cup First(F)$

$E \to g / \epsilon$ $\quad First(E) = \{g, \epsilon\}$

$F \to f / \epsilon$ $\quad First(F) = \{f, \epsilon\}$

$D \to EF$
$\epsilon F \Rightarrow F.(First)$

Ex: 2

$S \to (L) / a$ $\quad First(S) = \{(, a\}$

$L \to SL'$ $\quad First(L) = \{(, a\}$

$L' \to ,SL' / \epsilon$ $\quad First(L') = \{, , \epsilon\}$

## To Calculate Follow Set

Consider \$ in Follow(Start-Symbol), $\alpha$ - Terminal $Y, \beta$ - Non-terminal.

If the production is, $X \to \alpha\beta$

$Follow(\beta) = Follow(X)$

Search Non-terminal on RHS

If $X \to \alpha Y\beta$

$Follow(Y) = First(\beta)$ if $\epsilon$ is not in $First(\beta)$.

$= \{First(\beta)$ w/o $\epsilon\} \cup Follow(X)$

Ex: 1

$S \to aBDh$ | First(S) = {a} | Follow(S) = {$}
$B \to cC$ | First(B) = {c} | Follow(B) = {g, f, h}
$C \to bC / \epsilon$ | First(C) = {b, ε} | Follow(C) = {g, f, h}
$D \to EF$ | First(D) = {g, f, ε} | Follow(D) = {h}
$E \to g / \epsilon$ | First(E) = {g, ε} | Follow(E) = {f, h}
$F \to f / \epsilon$ | First(F) = {f, ε} | Follow(F) = {h}

$\not\epsilon F$
$g, f, \epsilon$

$B \underline{D} h$ $D \to EF$
$\not\epsilon h$ $E \not\epsilon \Rightarrow E$

Ex: 2

$S \to (L) / a$ | First(S) = {(, a} | Follow(S) = {$, , , )}
$L \to SL'$ | First(L) = {(, a} | Follow(L) = {)}
$L' \to , SL' / \epsilon$ | First(L') = {, , ε} | Follow(L') = {)}

$L \to SL'$
S · ε

Ex: 3

$E \to TE'$ | First(E) = {(, id} | Follow(E) = {$, )}
$E' \to +TE' / \epsilon$ | First(E') = {+, ε} | Follow(E') = {$, )}
$T \to FT'$ | First(T) = {(, id} | Follow(T) = {+, $, )}
$T' \to *FT' / \epsilon$ | First(T') = {*, ε} | Follow(T') = {+, $, )}
$F \to (E) / id$ | First(F) = {(, id} | Follow(F) = {*, +, $, )}

$E \to TE'$
$T \not\epsilon \to T$

$T \to FT'$
$F \not\epsilon$

# Predictive Parser (LL(1)) Parser

→ Top Down Parser

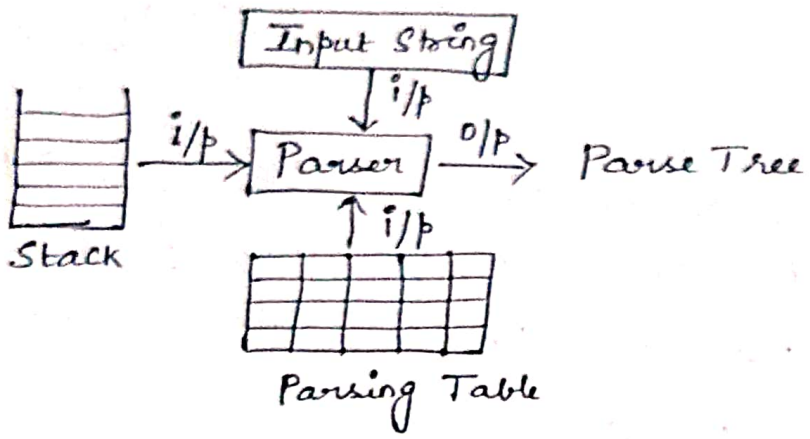
→ No Backtracking

→ LL(1) Parser
- L → One Token (Symbol)
- L → Left Most Derivation

Parsing Input String from left

I/p → Context free Grammar

O/p → Parse Tree.




## Steps to produce Parse Tree :-

1. Elimination of Left recursion
2. Elimination of left factor
3. Find First & Follow
4. Construct Parsing table.
5. Parsing Input String.
6. Construct Parse Tree.

Ex:-

$E \rightarrow TE'$

$E' \rightarrow +TE' / \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' / \epsilon$

$F \rightarrow (E) / id$

→ No left recursion & left factor.

3. Find First & Follow:-

First(E) = {(, id}  Follow(E) = {$, )}
First(E') = {+, ε}  Follow(E') = {$, )}
First(T) = {(, id}  Follow(T) = {+, $, )}
First(T') = {*, ε}  Follow(T') = {+, $, )}
First(F) = {(, id}  Follow(F) = {*, +, $, )}

4. Constructing Parsing table

Row → Non-Terminals  column → Terminals.

| NT/T | + | * | ( | ) | id | $ |
|---|---|---|---|---|---|---|
| E | | | E→TE' | | E→TE' | |
| E' | E'→+TE' | | | E'→ε | | E'→ε |
| T | | | T→FT' | | T→FT' | |
| T' | T'→ε | T'→*FT' | | T'→ε | | T'→ε |
| F | | | F→(E) | | F→id | |

To construct Parse table if the production is of the form, If $A \to \alpha$

(i) if $\alpha == \epsilon$ → Follow(A)  M[A, Follow(A)]

(ii) if $\alpha != \epsilon$ → First(α)  M[A, First(A)]
↓ Row No  ↓ Col. No.

(i) $E \to TE'$ ⇒ $\alpha != \epsilon$ ⇒ First(TE') ⇒ {(, id}  M[E,(], M[E, id]
($E$ ↓ A, $TE'$ = α)

(ii) $E' \to +TE'$ ⇒ $\alpha != \epsilon$ ⇒ First(+TE') ⇒ {+}  M[E', +]

(iii) $E' \to \epsilon$ ⇒ Follow(E') ⇒ {$, )}  M[E', $], M[E', )]

(iv) $T \to FT'$ ⇒ $\alpha != \epsilon$ ⇒ First(FT') = {(, id}  M[T,(], M[T, id]

(v) $T' \to *FT'$ ⇒ $\alpha != \epsilon$ ⇒ First(*FT') = {*}  M[T', *]

(vi) $T' \to \epsilon$ ⇒ Follow(T') = {+, $, )}  M[T', +], M[T', $], M[T', )]

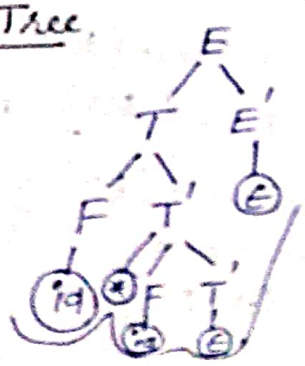
(vii) $F \to (E) \Rightarrow \alpha! = \epsilon \Rightarrow First((E)) \Rightarrow \{(\}$ $M[F,(]$

(viii) $F \to id \Rightarrow \alpha! = \epsilon \Rightarrow First(id) \Rightarrow \{id\}$ $M[F,id]$

5. Parsing input String I/p: id * id $ (Push the start symbol)

| Stack | Input String | Action |
|---|---|---|
| $ | id * id $ | — |
| $E | id * id $ | $E \to TE'$ (check for E & id in Parsing table) |
| $E'T | id * id $ | $T \to FT'$ |
| (Replace E with *TE') $E'T'F | id * id $ | $F \to id$ |
| $E'T'id | id * id $ | Pop (If both are terminals) |
| $E'T' | * id $ | $T' \to *FT'$ |
| $E'T'F* | * id $ | Pop |
| $E'T'F | id $ | $F \to id$ |
| $E'T'id | id $ | Pop |
| $E'T' | $ | $T' \to \epsilon$ |
| $E'\epsilon | $ | — |
| $E' | $ | $E' \to \epsilon$ |
| $\epsilon | $ | |
| $ | $ | Accept, halt, Success |

6. Construct Parse Tree.




id * id ε ε

id * id ε

$\boxed{id * id}$

**Ex: 2**

$S \to aABb$
$A \to c \mid \epsilon$
$B \to d \mid \epsilon$

I/p string: adb$

**Ex: 3**

$S \to (L) \mid a$
$L \to L,S \mid S$

I/p string: (a,a)

**Ex: 2**

1. No Left recursion & Left factoring
2. Find First & Follow.

First(S) = {a}  Follow(S) = {$}
First(A) = {c, ε}  Follow(A) = {d, b}
First(B) = {d, ε}  Follow(B) = {b}

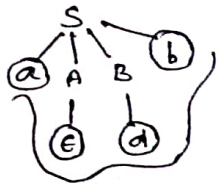
3. Constructing Parse Table

| NT/T | a | b | c | d | ε | $ |
|---|---|---|---|---|---|---|
| S | S→aABb | | S→aABb | | S→aABb | |
| A | | A→ε | A→c | | A→ε | |
| B | | B→ε | | B→d | | |

$S \to aABb \Rightarrow \{a\}$
$A \to c \Rightarrow \{c\}$
$A \to \epsilon \Rightarrow \{d, b\}$
$B \to d = \{d\}$
$B \to \epsilon = \{b\}$.

4. Parsing i/p string  i/p: adb$

| Stack | input | Action |
|---|---|---|
| $ | adb$ | — |
| $S | adb$ | S→aABb |
| $bBAa | adb$ | Pop |
| $bBA | db$ | A→ε |
| $bBε | db$. | B→d |
| $bd | db$ | Pop |
| $b | b$ | Pop |
| $ | $ | Accept, Halt, Success. |




aεdb
adb